

HUGH SARGONG'S ASSIGNMENT ON SPECIAL PYTHON METHODS

A notebook covering 16 Python special methods with simple examples.

1. `__init__` — Create an Object

Sets up the object when it is created.

```
In [ ]: class Fruit:
        def __init__(self, name, color):
            self.name = name
            self.color = color

        apple = Fruit('Apple', 'Red')
        print(apple.name, apple.color)
```

2. `__str__` — Print an Object

Shows a nice message when you print the object.

```
In [ ]: class Fruit:
        def __init__(self, name, color):
            self.name = name
            self.color = color

        def __str__(self):
            return f"A {self.color} {self.name}"

        apple = Fruit('Apple', 'Red')
        print(apple)
```

3. `__repr__` — Debug View

Shows the object in a way that helps developers.

```
In [ ]: class Fruit:
        def __init__(self, name, color):
            self.name = name
            self.color = color

        def __repr__(self):
            return f"Fruit('{self.name}', '{self.color}')"

        apple = Fruit('Apple', 'Red')
        print(repr(apple))
```

4. __len__ – Get Length

Lets len() work on your object.

```
In [ ]: class Basket:
        def __init__(self, fruits):
            self.fruits = fruits
        def __len__(self):
            return len(self.fruits)

b = Basket(['Apple', 'Banana', 'Mango'])
print(len(b))
```

5. __eq__ – Check Equality

Defines what == means for your object.

```
In [ ]: class Car:
        def __init__(self, model):
            self.model = model
        def __eq__(self, other):
            return self.model == other.model

c1 = Car('Toyota')
c2 = Car('Toyota')
c3 = Car('Honda')
print(c1 == c2)
print(c1 == c3)
```

6. __lt__ – Less Than

Lets you sort objects with <.

```
In [ ]: class Phone:
        def __init__(self, price):
            self.price = price
        def __lt__(self, other):
            return self.price < other.price
        def __repr__(self):
            return f"Phone({self.price})"

phones = [Phone(800), Phone(300), Phone(600)]
phones.sort()
print(phones)
```

7. __add__ – Add Objects

Teaches + how to work with your class.

```
In [ ]: class Time:
        def __init__(self, minutes):
            self.minutes = minutes
        def __add__(self, other):
            return Time(self.minutes + other.minutes)
        def __str__(self):
            return f"{self.minutes} min"

t1 = Time(30)
t2 = Time(45)
print(t1 + t2)
```

8. __getitem__ — Get by Index

Use [] to read items.

```
In [ ]: class Basket:
        def __init__(self, fruits):
            self.fruits = fruits
        def __getitem__(self, index):
            return self.fruits[index]

b = Basket(['Apple', 'Banana', 'Mango'])
print(b[0])
print(b[2])
```

9. __setitem__ — Set by Index

Use [] to change items.

```
In [ ]: class Shelf:
        def __init__(self, items):
            self.items = items
        def __setitem__(self, index, value):
            self.items[index] = value
        def __str__(self):
            return str(self.items)

s = Shelf(['Book', 'Pen', 'Ruler'])
s[1] = 'Eraser'
print(s)
```

10. __contains__ — Check Inside

Use in with your object.

```
In [ ]: class Basket:
        def __init__(self, fruits):
            self.fruits = fruits
        def __contains__(self, fruit):
            return fruit in self.fruits

b = Basket(['Apple', 'Banana', 'Mango'])
print('Apple' in b)
print('Grape' in b)
```

11. __call__ – Call Like a Function

Makes the object callable.

```
In [ ]: class Converter:
        def __init__(self, rate):
            self.rate = rate
        def __call__(self, amount):
            return amount * self.rate

usd_to_ghs = Converter(15.5)
print(usd_to_ghs(10))
print(usd_to_ghs(50))
```

12. __iter__ & __next__ – Loop Over

Lets for loops work.

```
In [ ]: class Numbers:
        def __init__(self, limit):
            self.limit = limit
            self.current = 0
        def __iter__(self):
            self.current = 0
            return self
        def __next__(self):
            if self.current >= self.limit:
                raise StopIteration
            num = self.current
            self.current += 1
            return num

for n in Numbers(5):
    print(n)
```

13. __bool__ – True or False

Controls if checks.

```
In [ ]: class Cup:
        def __init__(self, water):
            self.water = water
        def __bool__(self):
            return self.water > 0

empty = Cup(0)
full = Cup(200)
print(bool(empty))
print(bool(full))
```

14. __del__ – Cleanup

Runs when object is deleted.

```
In [ ]: class Room:
        def __init__(self, name):
            self.name = name
            print(f"Room {name} opened.")
        def __del__(self):
            print(f"Room {self.name} closed.")

r = Room('101')
del r
```

15. __enter__ & __exit__ – With Block

Use with with for safe setup and cleanup.

```
In [ ]: class Light:
        def __enter__(self):
            print('Light ON')
            return self
        def __exit__(self, a, b, c):
            print('Light OFF')

with Light():
    print('Reading...')
```

16. __hash__ – Use in Sets and Dicts

Lets object be a dict key or set item.

```
In [ ]: class Player:
        def __init__(self, id, name):
            self.id = id
```

```

        self.name = name

    def __eq__(self, other):
        return self.id == other.id

    def __hash__(self):
        return hash(self.id)

    def __repr__(self):
        return self.name

```

```

p1 = Player(1, 'John')
p2 = Player(1, 'John')
p3 = Player(2, 'Jane')

```

```

team = {p1, p2, p3}
print(team)

```

Summary Table

Method	Purpose	Trigger
<code>__init__</code>	Create object	<code>Class()</code>
<code>__str__</code>	Print nicely	<code>print(obj)</code>
<code>__repr__</code>	Debug view	<code>repr(obj)</code>
<code>__len__</code>	Count items	<code>len(obj)</code>
<code>__eq__</code>	Equal check	<code>==</code>
<code>__lt__</code>	Less than	<code><</code>
<code>__add__</code>	Add objects	<code>+</code>
<code>__getitem__</code>	Read index	<code>obj[i]</code>
<code>__setitem__</code>	Write index	<code>obj[i]=x</code>
<code>__contains__</code>	Check inside	<code>in</code>
<code>__call__</code>	Call object	<code>obj()</code>
<code>__iter__</code> / <code>__next__</code>	Loop	<code>for</code>
<code>__bool__</code>	True/False	<code>if</code>
<code>__del__</code>	Delete	<code>del</code>
<code>__enter__</code> / <code>__exit__</code>	Safe block	<code>with</code>
<code>__hash__</code>	Hash	<code>set, dict</code>